

Resultado ejecutivo

Componente	Score	Evaluación
README de la rama backend	8,0/10	Claro y técnicamente sólido, pero no demuestra seguridad, validación clínica ni pruebas
Backend MediSync descrito	6,2/10	Buena arquitectura, pero excesiva dependencia de Claude para decisiones clínicas
Frontend Angular descrito	4,5/10	Solo se menciona la tecnología; el README no demuestra funcionalidades ni UX
Frontend local HealthOS	7,2/10	Mockup profesional y funcional, aunque existen módulos incompletos y duplicación de interfaces
Backend local HealthOS	7,4/10	MVP seguro y explicable, pero todavía basado en CSV, tokens demo y poca persistencia
Fullstack local HealthOS	7,0/10	Buena demostración vendible; aún no es una plataforma clínica productiva
Preparación para piloto CESFAM/hospital	5,8/10	Apta para demostración sintética, no todavía para datos o decisiones reales
Preparación comercial/escalable	5,5/10	Falta arquitectura productiva, evidencia de impacto y operación multiinstitucional

Evaluación del README de la rama **backend**

Criterio	Score	Evidencia	Mejora propuesta
Explicación del problema	8	Describe priorización y coordinación de agenda	Incorporar cifras basales, población objetivo y KPI de reducción de espera
Flujo operacional	9	Explica diez estados desde derivación hasta cierre	Añadir cancelación, inasistencia, reagendamiento, duplicados y fallecimiento
Arquitectura	8	Clean Architecture, CQRS, MongoDB y Angular	Agregar diagrama de despliegue, límites de servicios y contratos OpenAPI

Instalación	8	Incluye Docker, comandos y puertos	Añadir script único de instalación y verificación automática
Reproducibilidad	7	Incluye 24 pacientes sintéticos	Publicar semilla, versión del dataset y resultados esperados de cada prueba
Evidencia funcional	6	Presenta cur1 , pero no resultados verificables	Añadir capturas, respuestas esperadas y estado de CI
Seguridad	4	No documenta autenticación, RBAC o segregación institucional	Incorporar OIDC, RBAC, auditoría, gestión de secretos y threat model
Privacidad	3	Se registran justificaciones, tool calls y prompts	Prohibir PHI en prompts, seudonimizar datos y definir retención y borrado
Validación clínica	3	HbA1c, glicemia y comorbilidades alimentan agentes IA	Documentar reglas, fuentes clínicas, calibración, sensibilidad y revisión médica
Resiliencia	3	Sin API key los agentes responden 502	Implementar reglas determinísticas, colas, reintentos y circuit breaker
Pruebas	4	El README no muestra suite ni cobertura	Incorporar unitarias, integración, seguridad, concurrencia y pruebas E2E
Escalabilidad	5	MongoDB y capas separadas ayudan	Agregar índices, paginación, procesamiento asíncrono y pruebas con millones de registros
Interoperabilidad Chile	3	No se documentan FHIR, TEI ni conectores locales	Definir recursos FHIR, terminologías y adaptadores on-premise autorizados
Documentación de limitaciones	6	Reconoce dependencia de Anthropic	Añadir limitaciones clínicas, regulatorias, operativas y epidemiológicas

La documentación es buena como PoC técnica, pero el README evidencia una solución todavía experimental.

Backend de la rama [backend](#)

Fortalezas

- Separación Domain, Application, Infrastructure, API e IA.
- Flujo de estados explícito y trazable.
- Revisión humana antes del agendamiento.
- Registro de justificaciones y consumo de tokens.
- Seeder y ambiente reproducible mediante Docker.

Riesgos principales

1. Claude calcula riesgo y prioridad — 4/10.

Una salida probabilística de un LLM no debería determinar directamente un orden clínico.

Mejora: motor determinístico, versionado y validado para el score; Claude solamente explica, resume antecedentes o detecta inconsistencias.

2. Disponibilidad — 3/10.

La falta de API key o caída externa interrumpe el flujo con 502.

Mejora: fallback determinístico, cola de tareas, reintentos limitados y ejecución diferida.

3. Seguridad y aislamiento — 4/10.

El README no demuestra usuarios, permisos ni aislamiento CESFAM/hospital.

Mejora: autenticación institucional, RBAC/ABAC, ámbito por establecimiento y auditoría inmutable.

4. Agendamiento automático — 6/10.

Reservar inmediatamente después de la revisión puede producir choques, cupos fantasma o inasistencias.

Mejora: ofrecer cupos, validar contactabilidad, confirmar al paciente y reservar con expiración.

5. Concurrencia — 5/10.

MongoDB necesita proteger transiciones y reservas simultáneas.

Mejora: versionado optimista, claves de idempotencia, transacciones y restricciones de agenda.

6. Representatividad — 3/10.

Veinticuatro pacientes sirven para demo, no para evaluar desempeño epidemiológico.

Mejora: generador reproducible de 10.000–100.000 casos más simulación agregada de 2,5 millones.

Frontend local HealthOS

Código evaluado:

[App.js](C:/Users/sandy/OneDrive/Documentos/HealthOS/src/frontend/src/App.js) y [desktop_web](C:/Users/sandy/OneDrive/Documentos/HealthOS/src/desktop_web/index.html).

Fortalezas

- Diseño coherente con una torre de control sanitaria.
- Roles de jefatura, médico y enfermería.
- Lista priorizada, explicación, simulador, epidemiología, IA y chatbot.
- Diferencia priorización de aumento de capacidad.
- Presenta DM2 multiespecialidad, no solamente pie diabético.
- Exportación CSV y selección de pacientes.

Problemas y mejoras

Problema	Score	Mejora
Existen frontend React y frontend desktop vanilla separados	5	Elegir una interfaz canónica y generar la versión desktop desde ella
Varios menús muestran contenido estructural genérico	6	Implementar pantallas reales para seguridad, usuarios, configuración y auditoría
Tabla seleccionable solo con mouse	6	Agregar navegación por teclado, foco visible y roles accesibles
Diseño muy denso para CESFAM	7	Crear interfaz simplificada por tareas para contraloría APS
Diseño hospitalario y CESFAM comparten demasiados elementos	6	Separar flujos, indicadores y permisos por edición
Falta estado clínico completo del caso	6	Mostrar derivación, revisión, contacto, agenda, confirmación y cierre
No existe gestión visual robusta de errores/reintentos	6	Estados offline, recuperación, retry y degradación controlada
Riesgo XSS en chatbot desktop	3	El mensaje ingresado se inserta mediante HTML; debe renderizarse como texto escapado
Accesibilidad no verificada	5	Auditoría WCAG 2.2 AA con axe y pruebas de teclado
No existen pruebas frontend visibles	4	Vitest, React Testing Library y Playwright para los recorridos centrales

Backend local HealthOS

Código principal:

[desktop_server.py](C:/Users/sandy/OneDrive/Documentos/HealthOS/src/backend/desktop_server.py) y

[prioritization_engine.py](C:/Users/sandy/OneDrive/Documentos/HealthOS/src/backend/prioritization_engine.py).

Fortalezas

- Motor determinístico y explicable.
- Separación entre priorizar y agregar capacidad.
- Datos sintéticos claramente identificados.
- RBAC básico.
- Chatbot sin text-to-SQL.
- Rechazo de URLs externas en visión artificial.
- Rate limiting, cabeceras de seguridad y redacción básica.
- Once pruebas locales existentes.
- Modelo epidemiológico agregado de aproximadamente 2,5 millones y muestra operacional de 10.000 registros.

Problemas y mejoras

Problema	Score	Mejora
Persistencia principal en CSV	5	PostgreSQL productivo con migraciones, índices y aislamiento institucional
Token demo emitido directamente por la aplicación	3	OIDC/OAuth2, MFA para administradores y sesiones revocables
Secreto JWT de desarrollo predeterminado	3	Obligar secreto externo y evitar arranque productivo con valor por defecto
Auditoría sin persistencia inmutable	5	Event log append-only, firma y exportación a SIEM
No existe workflow completo del caso	5	Máquina de estados con derivación, revisión, agenda, contacto y cierre
Once pruebas son insuficientes	5	Cobertura de autorización, fraude de token, concurrencia, contratos y E2E
Bridge Rayen/FHIR es demostrativo	4	Contrato formal de adaptadores, validación FHIR y entorno sandbox
Simulador de capacidad simplificado	6	Colas por especialidad, ausentismo, cancelaciones y capacidad variable
Modelo clínico no validado	4	Validación retrospectiva, calibración y monitoreo de sesgo
Observabilidad limitada	5	Métricas, trazas, alertas, SLO y dashboard operacional

Duplicación entre gateway y servidor desktop

6

Extraer servicios de dominio compartidos y eliminar endpoints duplicados

Recomendación fullstack

La mejor solución no es mantener MediSync y HealthOS como dos productos técnicos independientes.

La arquitectura objetivo debería combinar:

- **HealthOS** como marca y experiencia principal.
- Motor clínico determinístico de HealthOS.
- Máquina de estados, CQRS y flujo de agenda descritos por MediSync.
- Claude como copiloto explicativo, nunca como autoridad autónoma de prioridad.
- PostgreSQL como sistema transaccional.
- FHIR como contrato de interoperabilidad.
- Una sola interfaz responsive reutilizada para desktop y web.
- Ediciones configurables: **CESFAM** **Contralor** y **Hospital Operaciones**.
- Auditoría común de principio a fin.

Prioridad sugerida:

1. Corregir XSS, autenticación y secretos.
2. Unificar los dos frontend.
3. Implementar workflow y persistencia.
4. Integrar agenda/contactabilidad.
5. Ampliar pruebas y CI.
6. Validar clínicamente el score.
7. Realizar piloto sintético CESFAM–hospital.
8. Recién después conectar datos reales.